

ASSIGNMENT 6

Automation and Capacity Planning in a Containerized Microservices System

Site Reliability Engineering report following incident response and infrastructure provisioning work.

Docker Compose

Microservices

Prometheus

Grafana

Terraform

Capacity Planning

System: Frontend, auth-service, product-service, order-service, user-service, chat-service, MongoDB, Prometheus, Grafana.

Prepared: May 5, 2026

Repository evidence: source code, Docker configuration, monitoring rules, scripts, and screenshots are included in the project workspace.

1. Executive Summary

This assignment extends the previous incident response and Terraform infrastructure work by adding SRE automation, monitoring-based alerting, capacity testing, and scaling analysis. The system is a Dockerized microservices application with a frontend gateway, five backend services, a database container, and an observability stack.

The previous incident was caused by an invalid database connection configuration. The new automation reduces this risk through environment validation, Docker health checks, restart policies, and Prometheus alert rules.

1000 Load test requests sent to order-service health endpoint.	0 Failures during the load test.	1814.85 Requests per second observed by the local load-test script.
60.39 ms p95 latency for the load-test run.	7 Prometheus alert rules implemented.	Healthy Docker health checks confirmed for the application stack.

2. Criteria Coverage

Evaluation / Deliverable Item	Implementation in This Submission	Evidence
Automation implementation	Docker Compose startup, pre-deployment configuration validation, Docker health checks, and automatic restart policies.	<code>docker-compose.yml</code> , <code>scripts/start-stack.ps1</code> , <code>scripts/validate-env.ps1</code> , Figure 1.
Monitoring and alerting quality	Prometheus scrapes every service and loads alert rules for service downtime, unhealthy state, error rate, latency, CPU, memory, and restarts.	<code>prometheus/prometheus.yml</code> , <code>prometheus/alerts.yml</code> , Figures 4-6.
Capacity planning depth	Load simulation, RPS, error rate, latency, CPU, memory, restart metrics, and bottleneck analysis.	<code>scripts/load-test.mjs</code> , <code>scripts/prometheus-snapshot.ps1</code> , Figures 7-9.
Integration with previous assignments	Builds on Assignment 4 incident by validating database configuration and detecting service failure. Builds on Assignment 5 by keeping Terraform as the infrastructure layer.	Sections 5 and 8, incident and recovery screenshots.
Scalability and resilience demonstration	Recovery is demonstrated by stopping order-service, observing target failure and a firing alert, then restarting the service. Scaling strategies are proposed.	Figures 10-14, Section 8.

Scope note: The assignment context mentions PostgreSQL. This project uses MongoDB as the existing database layer from the previous microservices implementation. The SRE mechanisms apply to the database dependency in the same way: health checks, configuration validation, incident simulation, and capacity analysis focus on the database-backed services.

3. System Context

The system contains the following Dockerized components:

- **Frontend:** Nginx-based gateway and static frontend on port `8080`.
- **auth-service:** Firebase authentication verification on port `4000`.
- **product-service:** Product and category API backed by MongoDB on port `4001`.
- **order-service:** Order creation and order history API backed by MongoDB on port `4002`.
- **user-service:** Profile/project API and internal calls to order-service on port `4003`.
- **chat-service:** User/admin messaging API backed by MongoDB on port `4004`.
- **Prometheus:** metrics and alerts on port `9090`.
- **Grafana:** dashboard visualization on port `3000`.
- **Terraform:** infrastructure provisioning for the Docker host and firewall rules.

4. Automation Implementation

4.1 Automated deployment

The full stack can be deployed using one PowerShell command:

```
.\scripts\start-stack.ps1
```

This script validates configuration first and then runs:

```
docker compose up -d --build
```

4.2 Configuration validation

The pre-deployment validation script checks the required configuration keys:

- `FIREBASE_PROJECT_ID`
- `FIREBASE_CREDENTIALS_HOST_PATH`
- `CORS_ORIGINS`
- `MONGO_URL`

It also rejects the known incident simulation value `mongodb://wrong-mongodb:27017/shop`, which helps prevent a repeat of the earlier database misconfiguration incident.

4.3 Health checks and restart policies

All backend services expose `/health` and `/metrics`. Docker Compose includes health checks and `restart: unless-stopped` policies. This gives the system automatic failure detection at the container level and restart recovery when a service exits unexpectedly.

```
healthcheck:
  test: ["CMD-SHELL", "node -e ... /health ..."]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 20s

restart: unless-stopped
```

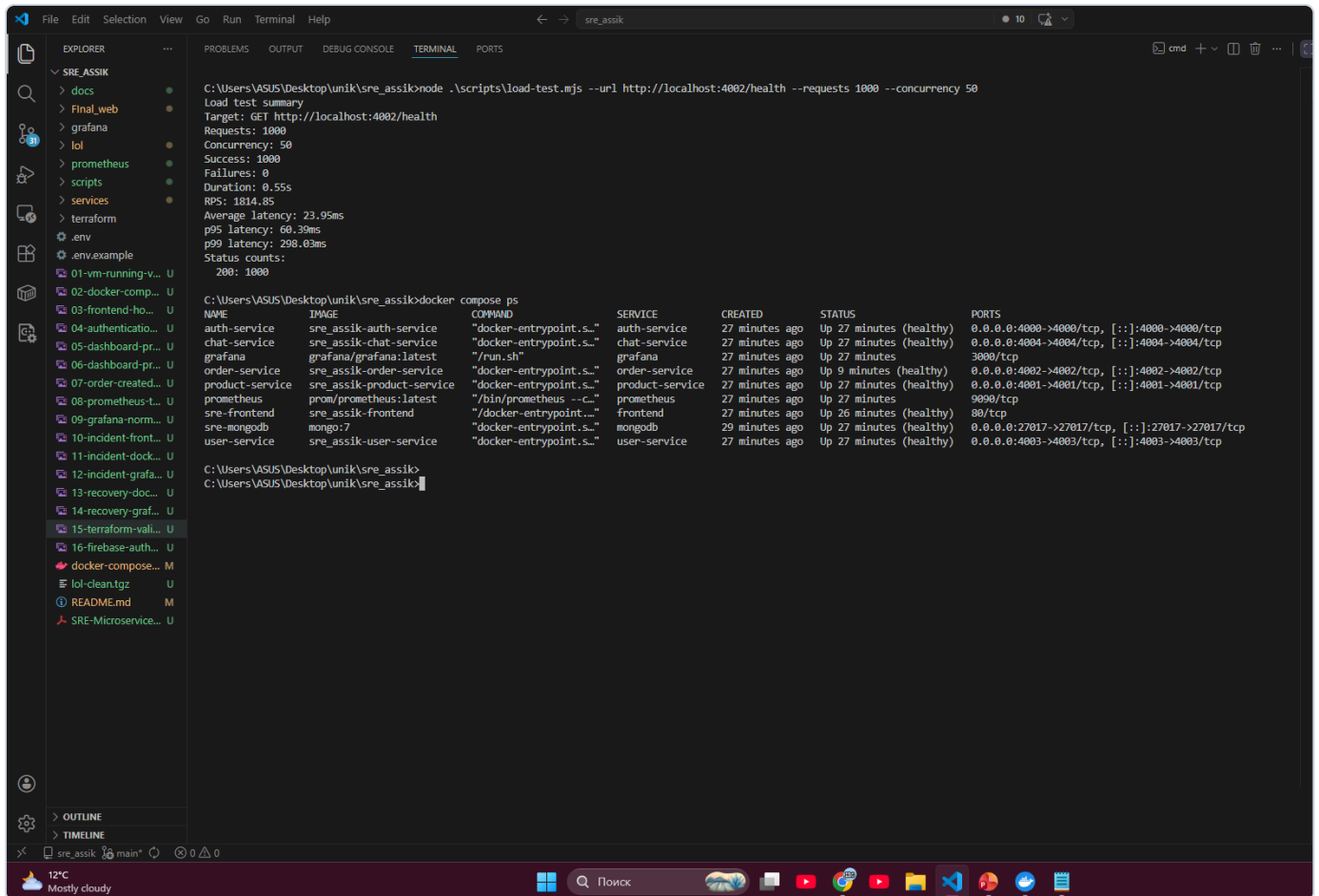


Figure 1. Docker Compose confirms that the application containers are running and healthy.

5. Health Check Evidence

The service health endpoints return JSON status responses. Database-backed services also report the database connection state. This supports both manual troubleshooting and automated container health checks.

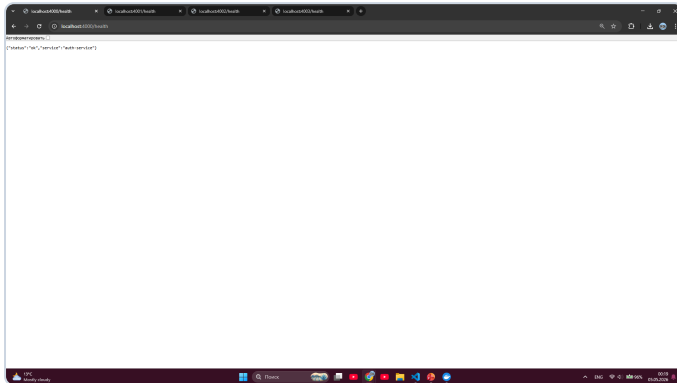


Figure 2a. auth-service returns `status: ok`.

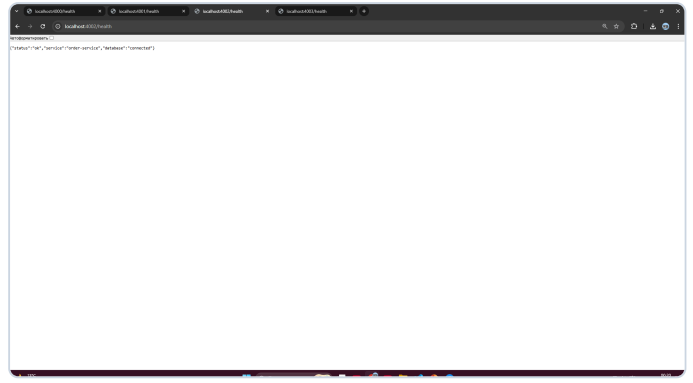


Figure 2b. order-service returns `status: ok` and `database: connected`.

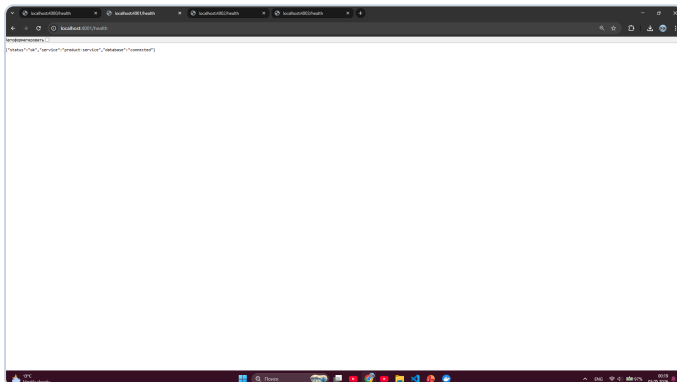


Figure 2c. product-service returns `status: ok` and `database: connected`.

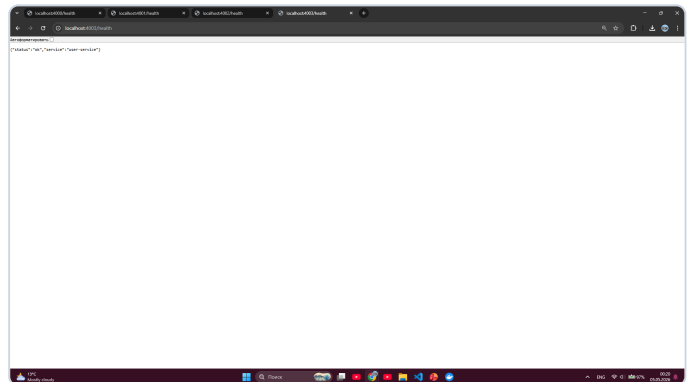


Figure 2d. user-service returns `status: ok`.

6. Monitoring and Alerting

Prometheus is configured with a 5-second scrape interval and collects metrics from each backend service.

```
rule_files:
  - /etc/prometheus/alerts.yml
```

Implemented alert rules:

- `ServiceDown` : Prometheus cannot scrape a service.
- `ServiceUnhealthy` : a service health metric reports unhealthy state.
- `HighErrorRate` : more than 5 percent recent HTTP responses are errors.
- `HighLatency` : p95 latency is above 1 second.
- `HighProcessCpu` : a service process uses more than 0.8 CPU cores.
- `HighProcessMemory` : a service process uses more than 300 MB resident memory.
- `ServiceRestartedRecently` : a service process start time changed within 10 minutes.

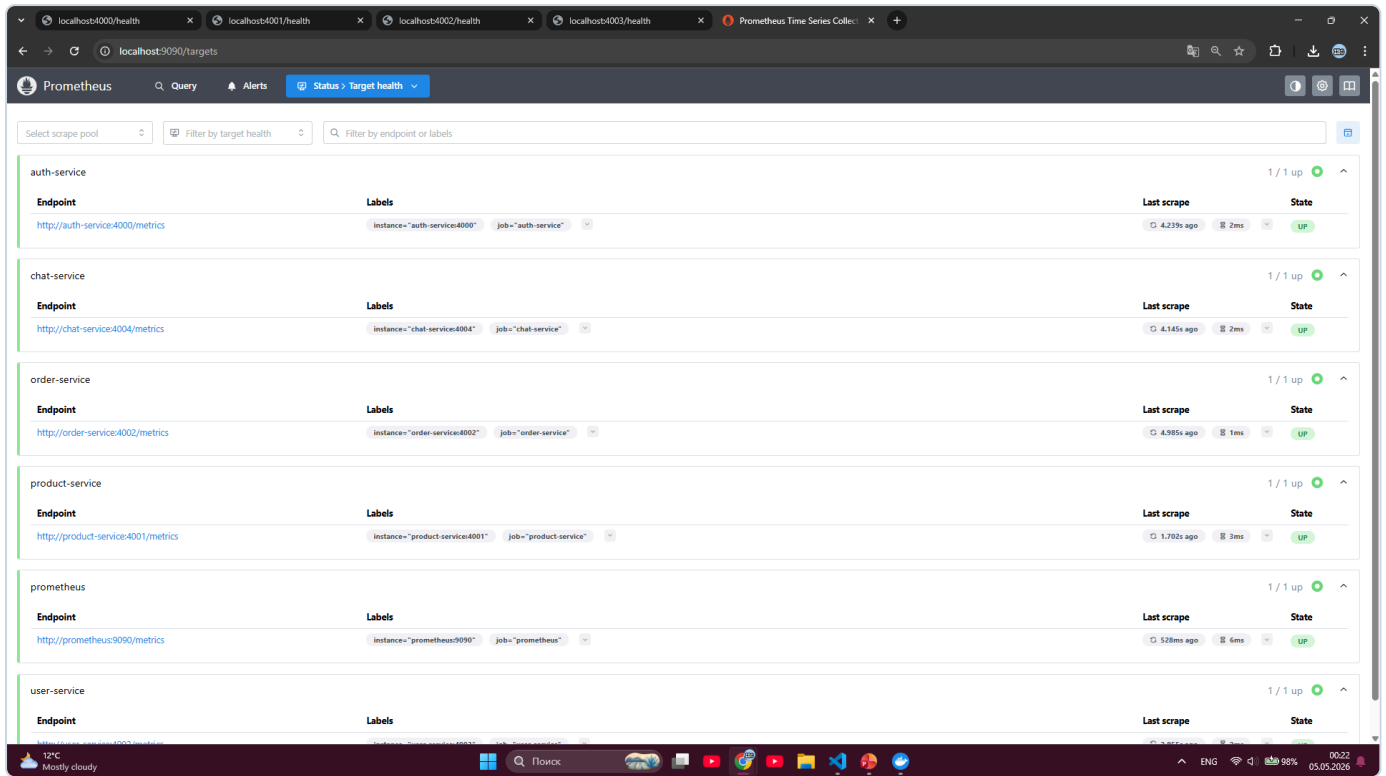


Figure 3. Prometheus target health page shows service metric endpoints in the **UP** state.

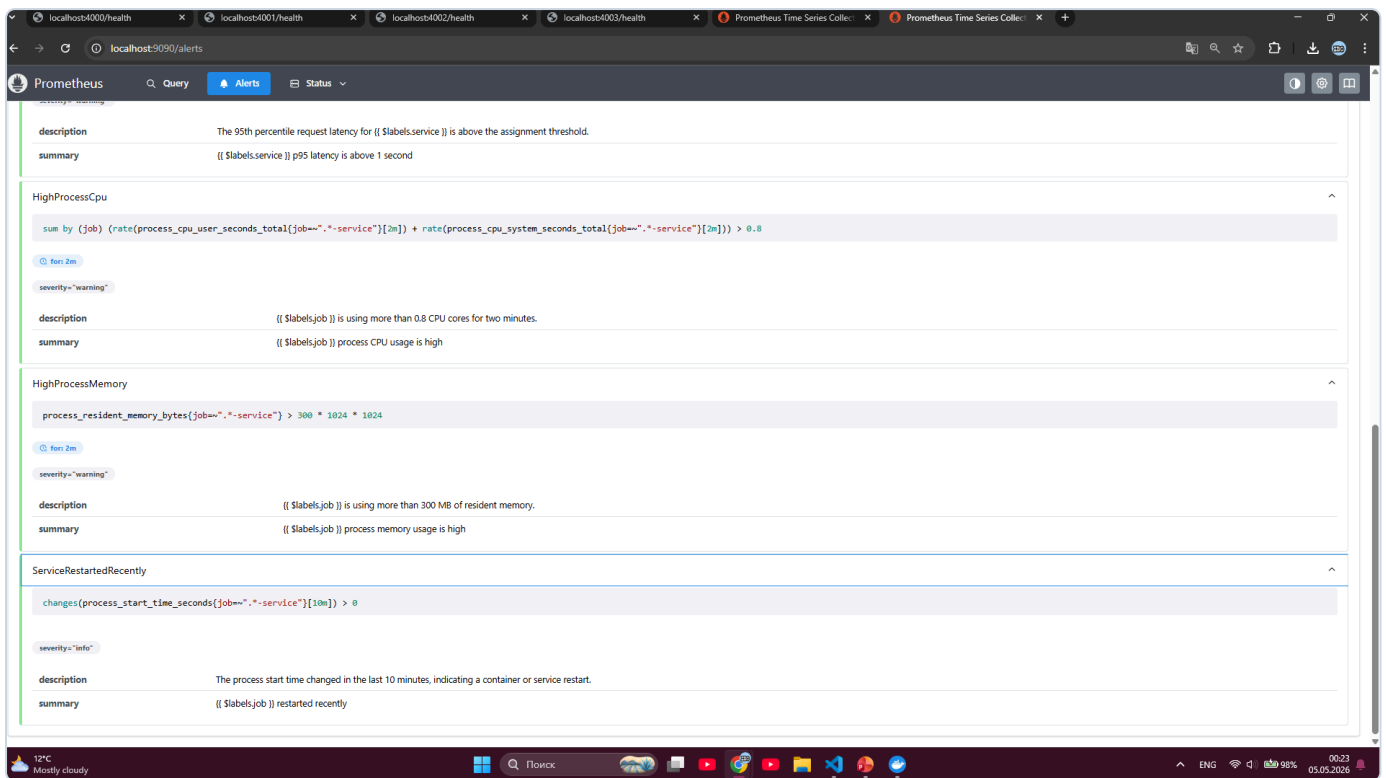


Figure 4. Alert rules for service downtime, unhealthy state, high error rate, and high latency.

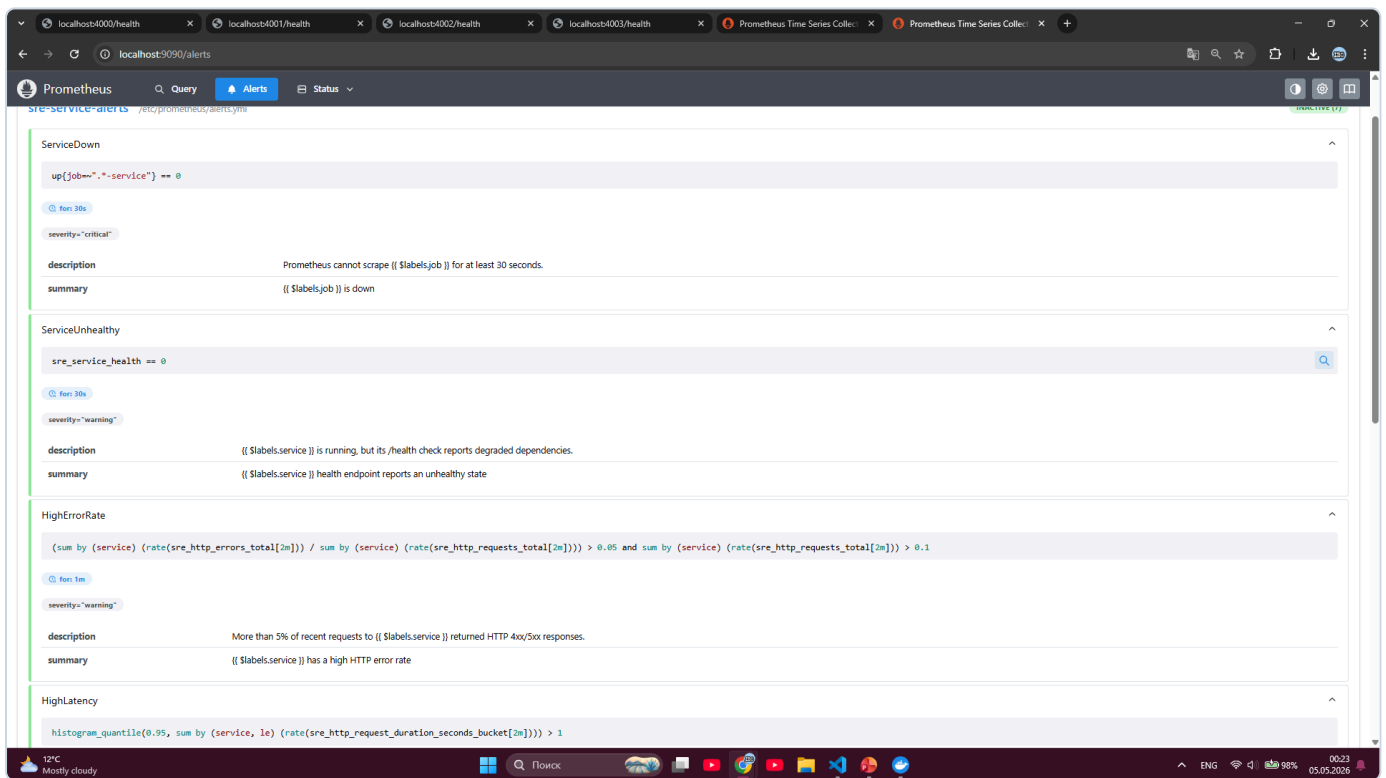


Figure 5. Alert rules for high process CPU, high process memory, and recent service restarts.

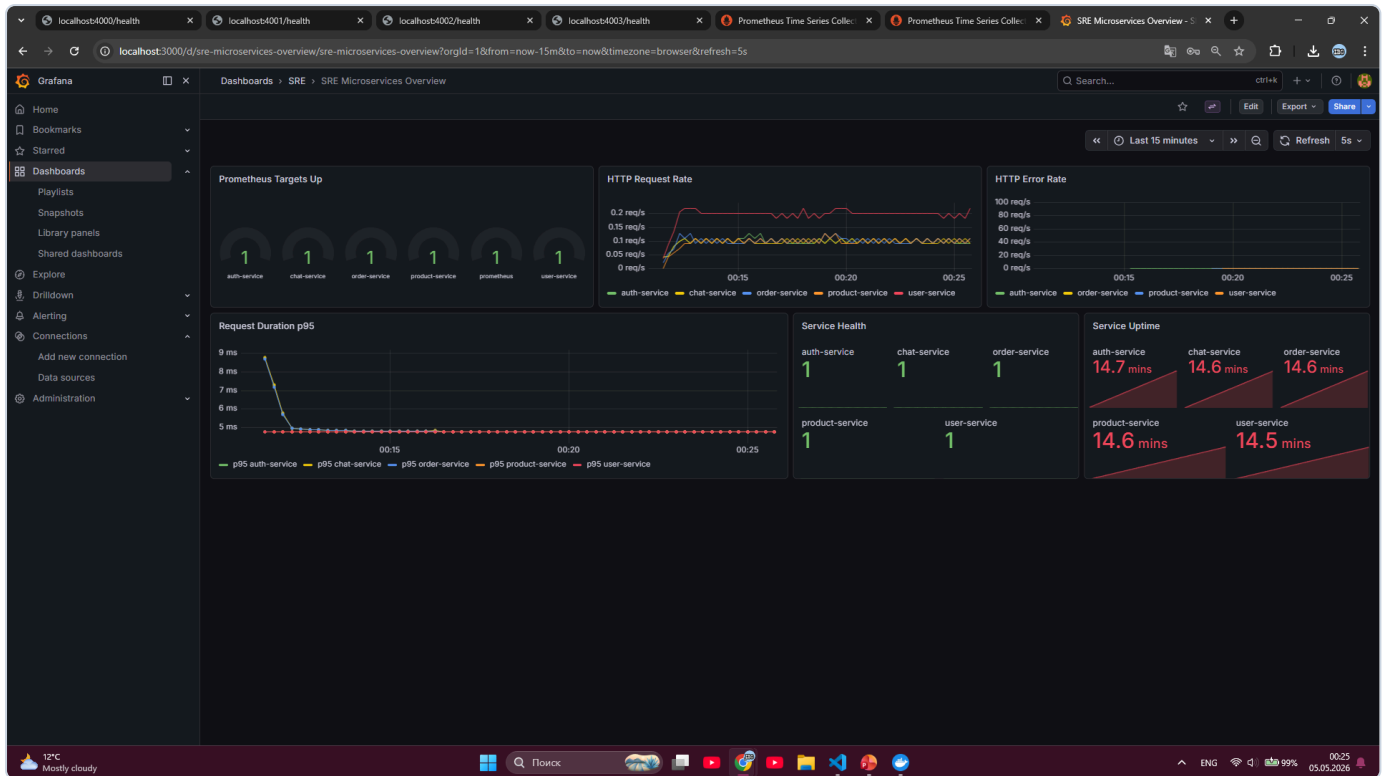


Figure 6. Grafana dashboard during normal operation: targets are up, error rate is zero, and service health values are 1.

7. Capacity Planning

7.1 Load simulation method

A Node.js load-test script was added to generate concurrent requests and report request rate, success count, failure count, average latency, p95 latency, and p99 latency.

```
node .\scripts\load-test.mjs --url http://localhost:4002/health --requests 1000 --concurrency 50
```

The load test focused on `order-service` because this service is the main capacity risk: it owns order workflows and depends on the database layer. A health endpoint was used for repeatable local load generation without requiring live Firebase user tokens.

Production-like testing of `POST /api/orders` and `GET /api/orders/my` requires valid Firebase ID tokens and realistic product/user data. For this assignment, the `/health` endpoint was used as a controlled baseline to measure service availability, request handling, latency, and observability behavior without introducing authentication-token setup as a separate variable. Therefore, the result should be interpreted as a local baseline capacity test, not as the absolute production capacity of the order workflow.

7.2 Load test results

Metric	Observed Value	Interpretation
Total requests	1000	The test generated enough traffic to create a clear request-rate spike.
Concurrency	50	Requests were sent concurrently instead of one by one.
Success / Failures	1000 success / 0 failures	The service stayed available during the test.
Local script RPS	1814.85 requests/second	The lightweight health endpoint handled local traffic without saturation.
Average latency	23.95 ms	Average response time remained low.
p95 latency	60.39 ms	95 percent of requests completed below this threshold.
p99 latency	298.03 ms	A small number of requests were slower, but still below the 1 second alert threshold.

The screenshot shows a Visual Studio Code window with a terminal open. The terminal displays the output of a load test command. The command executed is `C:\Users\ASUS\Desktop\unik\sre_assik>node .\scripts\load-test.mjs --url http://localhost:4002/health --requests 1000 --concurrency 50`. The output is a JSON object with the following fields: `Load test summary`, `Target: GET http://localhost:4002/health`, `Requests: 1000`, `Concurrency: 50`, `Success: 1000`, `Failures: 0`, `Duration: 0.55s`, `RPS: 1814.85`, `Average latency: 23.95ms`, `p95 latency: 60.39ms`, `p99 latency: 298.03ms`, and `Status counts: 200: 1000`. The terminal also shows the current directory `C:\Users\ASUS\Desktop\unik\sre_assik`.

```
C:\Users\ASUS\Desktop\unik\sre_assik>node .\scripts\load-test.mjs --url http://localhost:4002/health --requests 1000 --concurrency 50
Load test summary
Target: GET http://localhost:4002/health
Requests: 1000
Concurrency: 50
Success: 1000
Failures: 0
Duration: 0.55s
RPS: 1814.85
Average latency: 23.95ms
p95 latency: 60.39ms
p99 latency: 298.03ms
Status counts:
  200: 1000

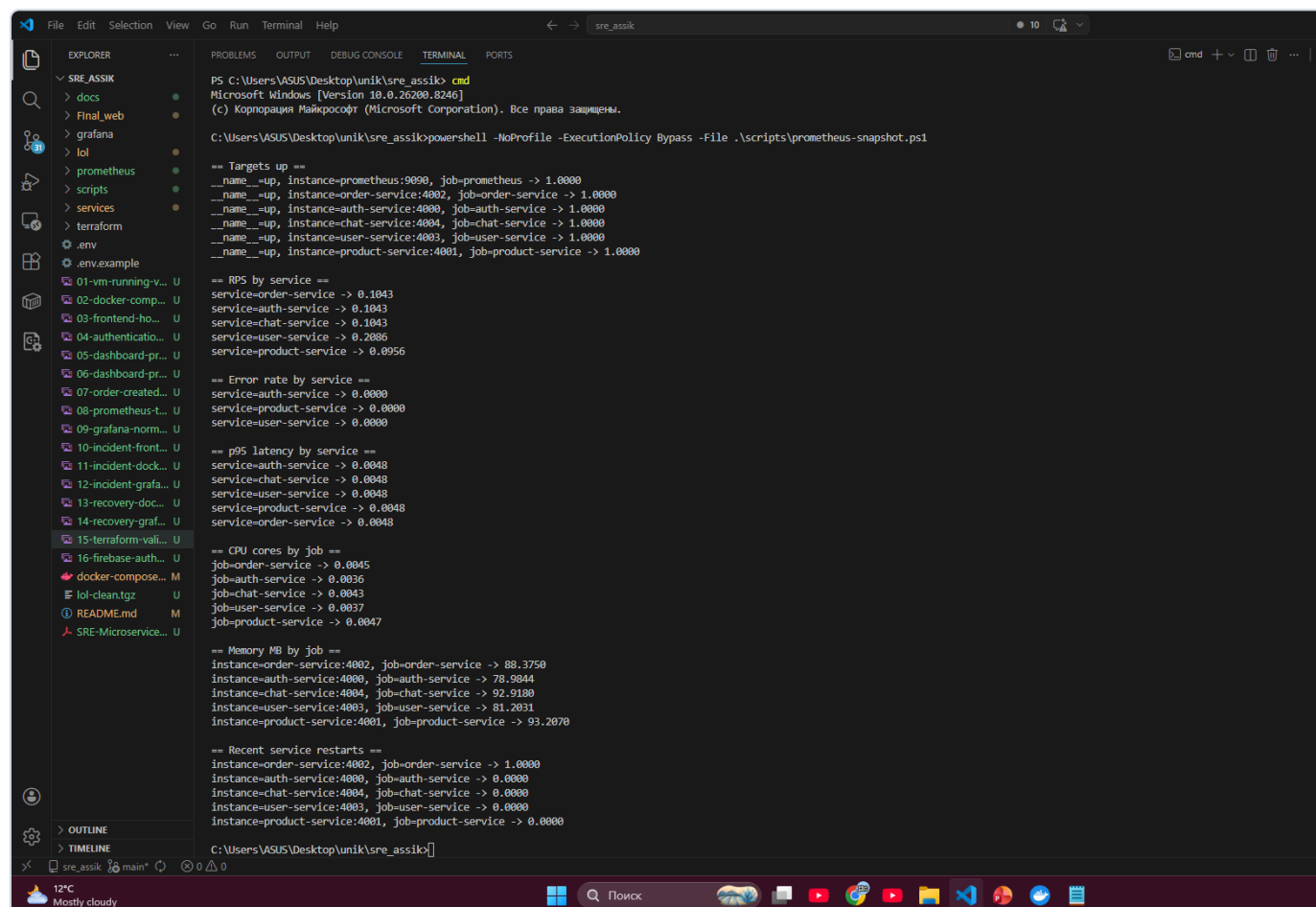
C:\Users\ASUS\Desktop\unik\sre_assik
```

Figure 7. Load-test output for order-service: 1000 requests, 0 failures, and measured latency percentiles.

7.3 Metrics collected after load

The Prometheus snapshot script collected target state, request rate, error rate, latency, CPU, memory, and restart metrics.

Metric Group	Observation	Capacity Meaning
Targets up	All listed service targets returned value 1.	The stack stayed observable after the load test.
Error rate	Visible services reported 0.0000 error rate.	No HTTP error increase was detected after the test.
p95 latency	Approximately 0.0048 seconds in the Prometheus snapshot.	Prometheus latency remained low after smoothing over the query window.
CPU	All service processes were below 0.01 CPU cores in the snapshot.	The local health test did not saturate CPU resources.
Memory	Services used about 79 MB to 93 MB resident memory.	Memory was stable and below the configured high-memory alert threshold.
Restart count	order-service showed 1 recent restart.	This reflects the incident/recovery test, not an unexpected crash.



```
PS C:\Users\ASUS\Desktop\unik\sre_assik> cmd
Microsoft Windows [Version 10.0.26200.8246]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\ASUS\Desktop\unik\sre_assik> powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\prometheus-snapshot.ps1

== Targets up ==
__name__=up, instance=prometheus:9090, job=prometheus -> 1.0000
__name__=up, instance=order-service:4002, job=order-service -> 1.0000
__name__=up, instance=auth-service:4000, job=auth-service -> 1.0000
__name__=up, instance=chat-service:4004, job=chat-service -> 1.0000
__name__=up, instance=user-service:4003, job=user-service -> 1.0000
__name__=up, instance=product-service:4001, job=product-service -> 1.0000

== RPS by service ==
service=order-service -> 0.1043
service=auth-service -> 0.1043
service=chat-service -> 0.1043
service=user-service -> 0.2086
service=product-service -> 0.0956

== Error rate by service ==
service=auth-service -> 0.0000
service=product-service -> 0.0000
service=user-service -> 0.0000

== p95 latency by service ==
service=auth-service -> 0.0048
service=chat-service -> 0.0048
service=user-service -> 0.0048
service=product-service -> 0.0048
service=order-service -> 0.0048

== CPU cores by job ==
job=order-service -> 0.0045
job=auth-service -> 0.0036
job=chat-service -> 0.0043
job=user-service -> 0.0037
job=product-service -> 0.0047

== Memory MB by job ==
instance=order-service:4002, job=order-service -> 88.3750
instance=auth-service:4000, job=auth-service -> 78.9844
instance=chat-service:4004, job=chat-service -> 92.9180
instance=user-service:4003, job=user-service -> 81.2031
instance=product-service:4001, job=product-service -> 93.2070

== Recent service restarts ==
instance=order-service:4002, job=order-service -> 1.0000
instance=auth-service:4000, job=auth-service -> 0.0000
instance=chat-service:4004, job=chat-service -> 0.0000
instance=user-service:4003, job=user-service -> 0.0000
instance=product-service:4001, job=product-service -> 0.0000

C:\Users\ASUS\Desktop\unik\sre_assik>
```

Figure 8. Prometheus snapshot after testing: RPS, error rate, p95 latency, CPU, memory, and restart data.

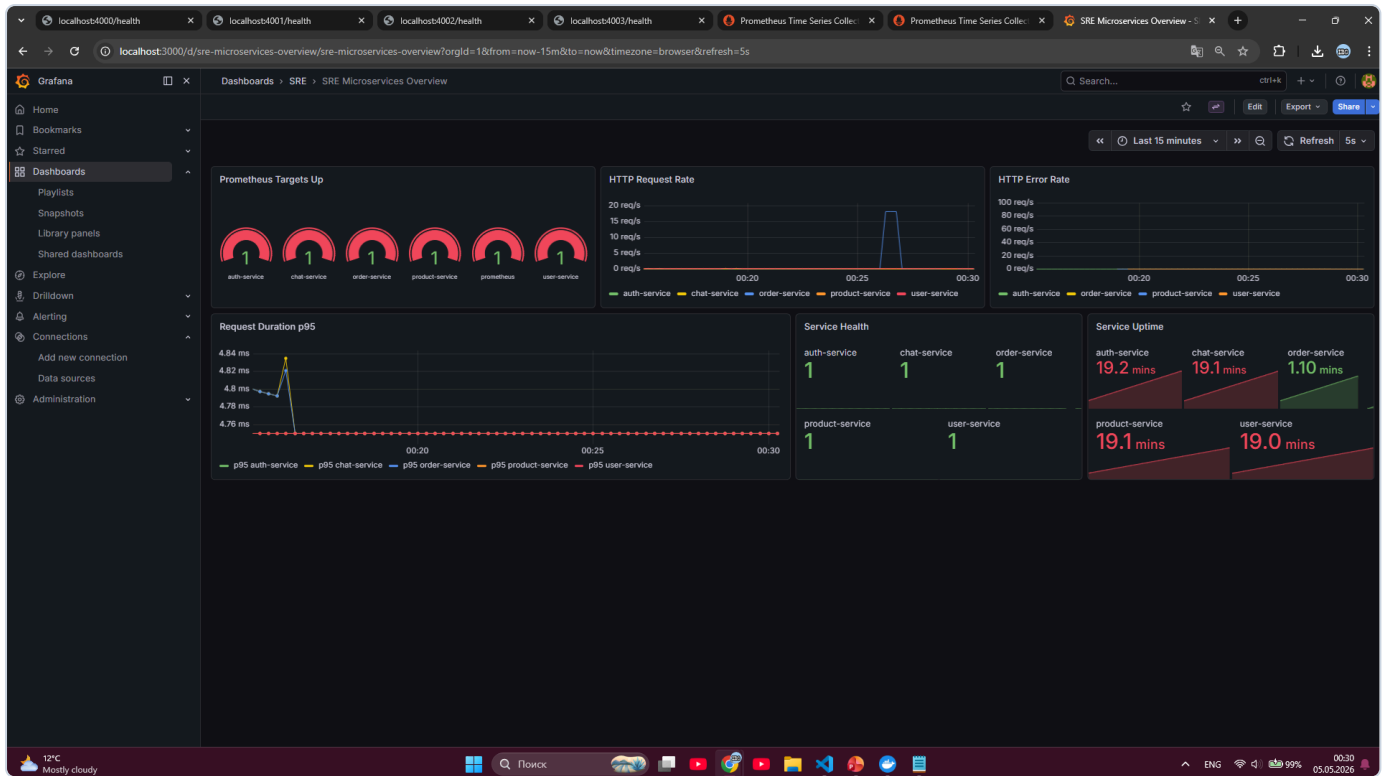


Figure 9. Grafana shows the request-rate spike for order-service and healthy service state after recovery.

8. Bottleneck and Capacity Analysis

The current local load test did not saturate CPU, memory, or latency thresholds. However, the architecture still has clear capacity risks:

- **order-service**: highest business-critical capacity risk because it owns order workflows and depends on MongoDB.
- **MongoDB**: shared dependency for product-service, order-service, and chat-service. A database slowdown can degrade several services at once.
- **Single-container services**: each backend service runs as one container, so a single service instance limits horizontal capacity.
- **Frontend gateway**: Nginx currently routes to single service names; future multi-instance routing should use an orchestrator or load balancer.

The main conclusion is that the system is reliable under the tested local health-check load, but production-like capacity testing should use authenticated order creation and order history requests with realistic user data.

9. Incident Recovery Demonstration

Recovery was tested by stopping `order-service`, observing Prometheus target failure, confirming a firing `ServiceDown` alert, then starting `order-service` again and verifying that targets returned to `UP`.

```
docker compose stop order-service
docker compose up -d order-service
```

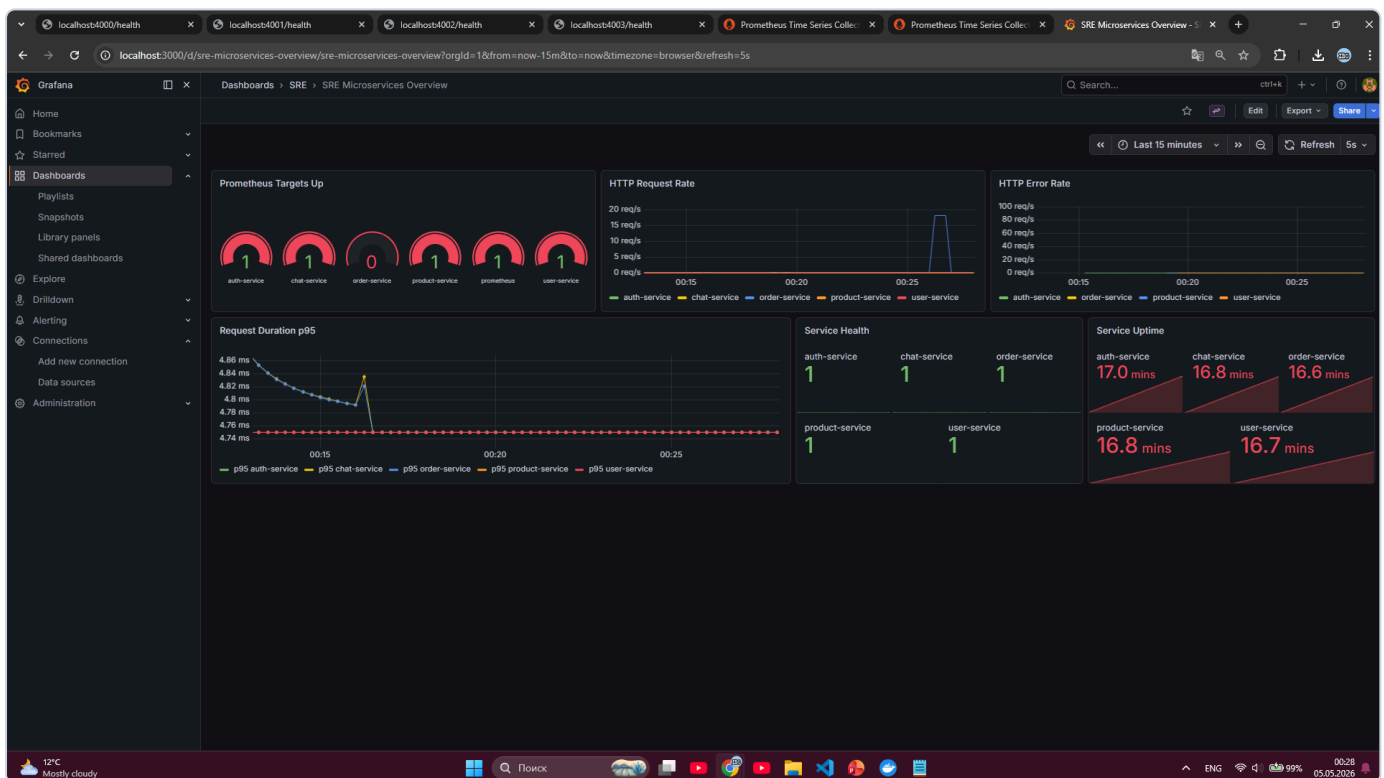


Figure 10. Grafana during the incident: order-service target is down while other services remain available.

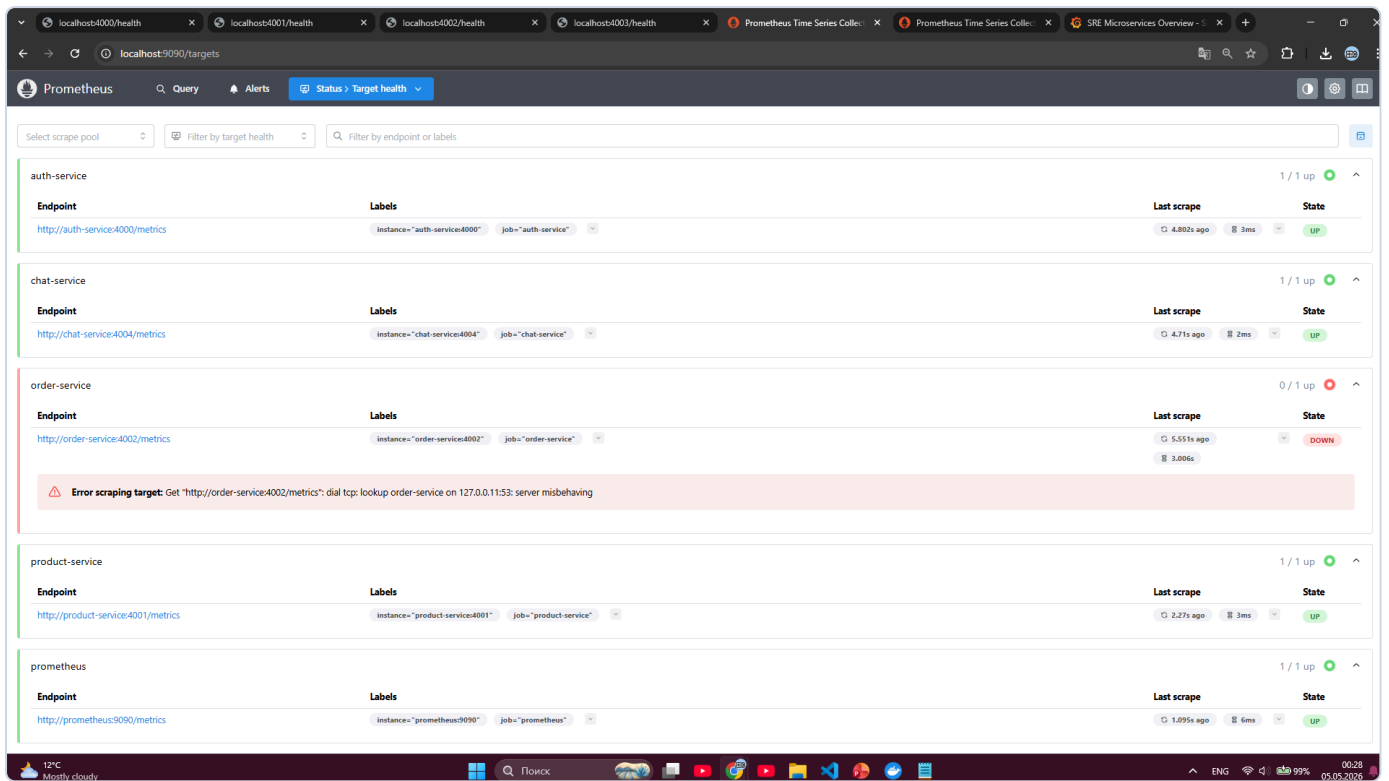


Figure 11. Prometheus target health shows order-service in the **DOWN** state.

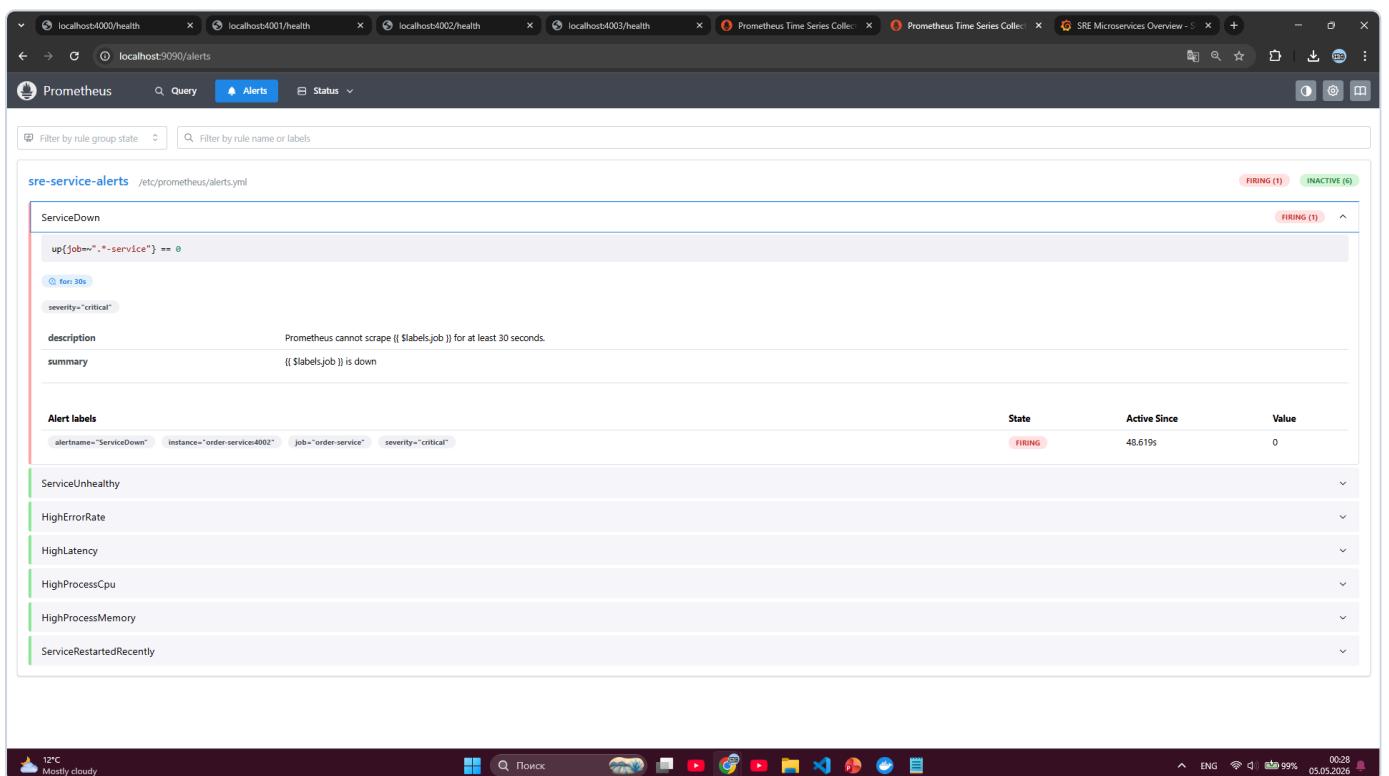


Figure 12. Prometheus alert **ServiceDown** is firing for order-service.

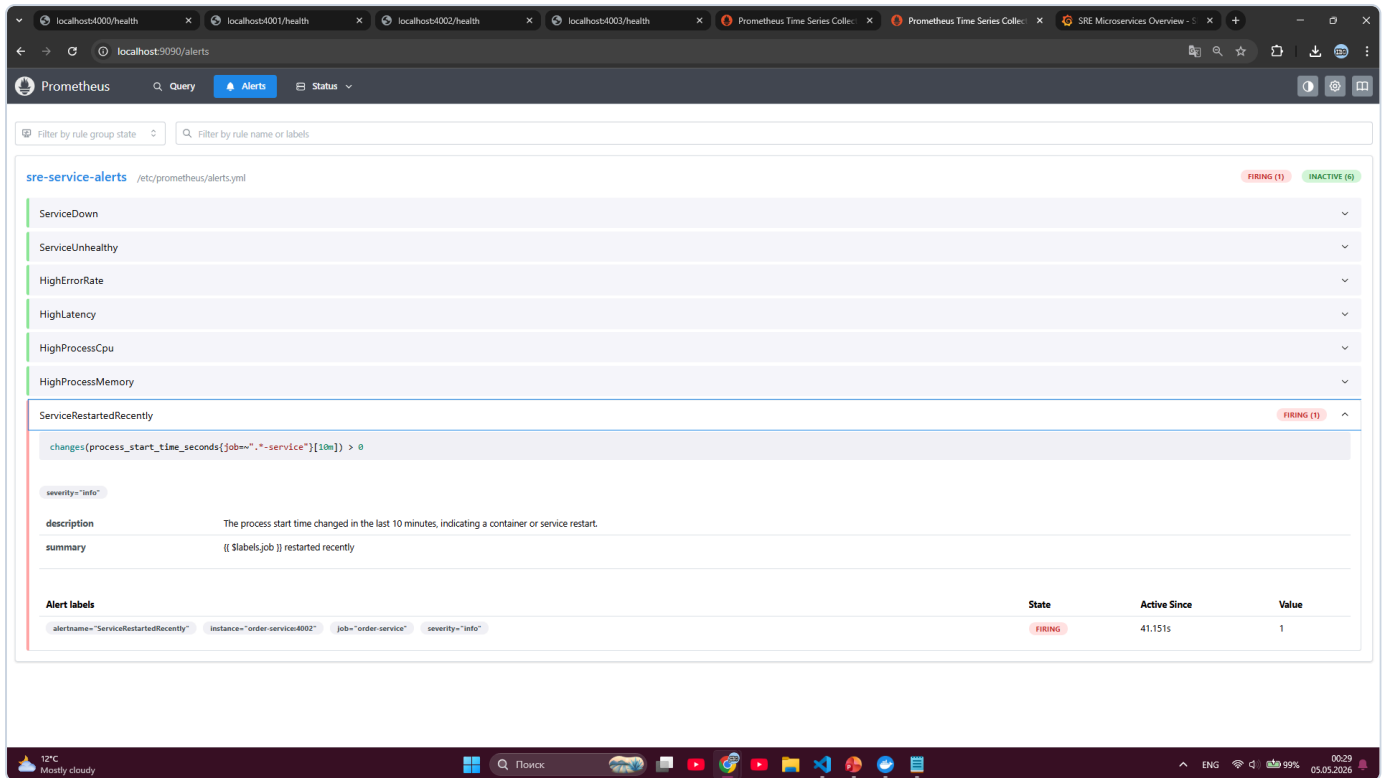


Figure 13. Prometheus detects that order-service restarted recently after recovery.

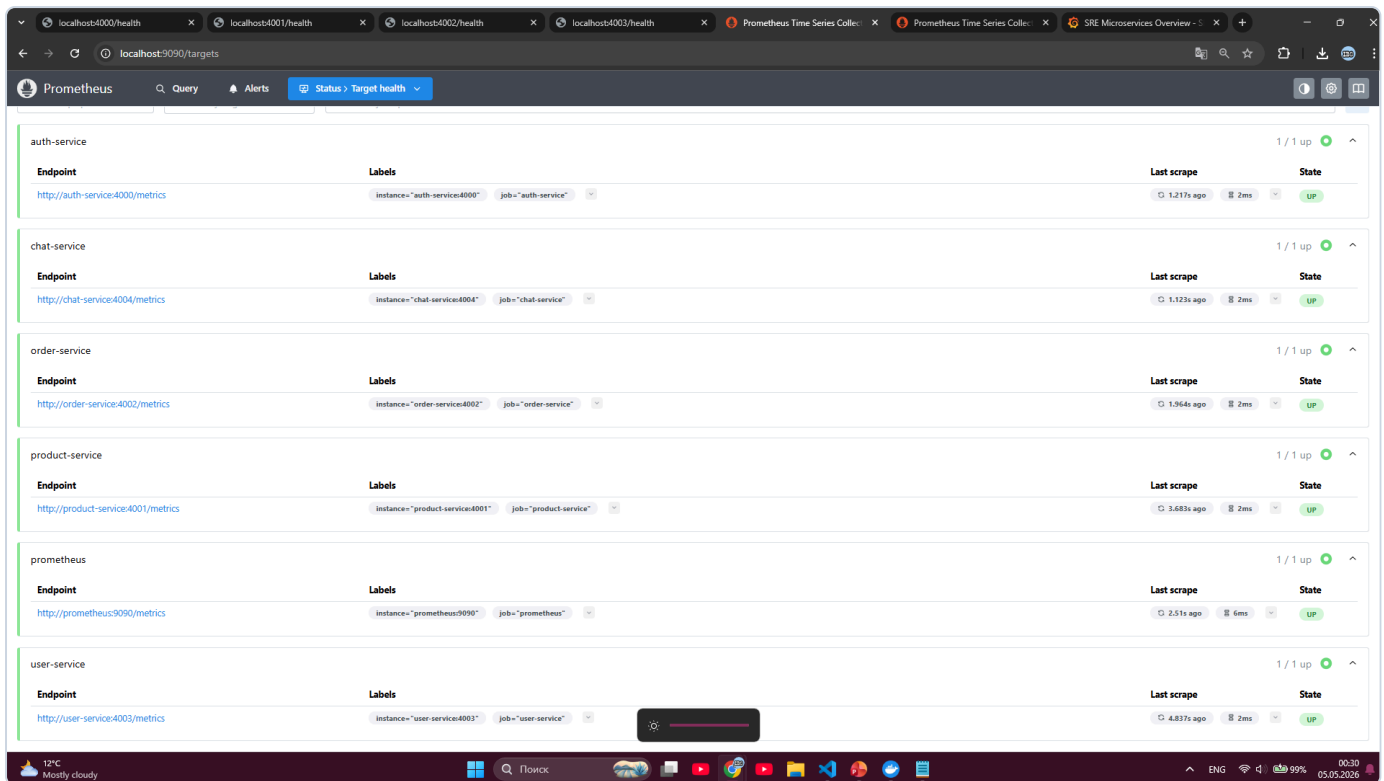


Figure 14. Recovery state: Prometheus target health shows all service targets back in the `UP` state.

10. Scaling Strategy

10.1 Horizontal scaling

The first horizontal scaling target should be `order-service`. Future deployment should run multiple order-service replicas and place a load balancer or orchestrator in front of them.

```
order-service-1
order-service-2
order-service-3
```

Possible implementation options:

- Docker Swarm service replicas with ingress routing.
- Kubernetes Deployment and Service objects.
- Nginx upstream load balancing for several backend instances.

10.2 Vertical scaling through Terraform

Terraform already owns VM infrastructure definitions. If load increases, the VM type can be increased through the Terraform variable `machine_type`.

```
machine_type = "e2-medium"
```

Future larger option:

```
machine_type = "e2-standard-2"
```

10.3 Database optimization

- Add indexes for common order queries, especially user order history by `userId` and creation time.
- Tune MongoDB CPU and memory resources or move the database to a managed service.
- Use connection pooling and sensible timeout settings.
- Separate read-heavy and write-heavy workloads if request volume grows.

11. Improvements Based on Previous Assignments

Previous Finding	Improvement Added in Assignment 6
Database misconfiguration caused service failure.	Pre-deployment validation checks <code>MONGO_URL</code> and rejects the known broken hostname.
Manual detection required checking containers and dashboards.	Prometheus alert rules detect service-down, unhealthy state, high errors, high latency, CPU, memory, and restarts.
Recovery depended on manual service restart.	Docker restart policies and health checks reduce manual recovery effort.
Infrastructure was defined with Terraform in Assignment 5.	Scaling strategy uses the existing Terraform <code>machine_type</code> variable for vertical scaling.

12. Limitations and Next Steps

The current implementation demonstrates automation, alerting, load simulation, and recovery. It does not fully implement production auto-scaling. Horizontal scaling, Kubernetes-based auto-scaling, and managed database scaling are proposed strategies rather than completed runtime features.

The load test is also intentionally limited to the `/health` endpoint. Authenticated order endpoints require Firebase ID tokens, so a production-like test would need a token generation or login flow, seeded product IDs, and realistic user scenarios.

Recommended next steps:

- Run an authenticated load test against real order creation and order history endpoints.
- Add container-level CPU and memory metrics with cAdvisor or Docker metrics exporter.
- Run multi-replica order-service behind a load balancer.
- Move database capacity testing to realistic product/order datasets.
- Integrate alert delivery through Alertmanager, email, Telegram, Slack, or another notification channel.

13. Conclusion

The system now includes practical SRE automation and observability improvements: validated configuration, health checks, automatic restart policies, Prometheus alert rules, Grafana dashboards, load testing, and recovery evidence. Capacity testing shows that the local order-service health endpoint remains stable under concurrent traffic with zero failures, while the architecture analysis identifies order-service and the shared database as the main capacity risks for future production-like workloads.